

函数式 EDA Demo

Haskell 技术主线说明

重点：用了哪些 Haskell 知识、这些知识是什么、为什么适合这个问题

一句话定位

这个项目不是生产级 EDA 工具，而是用一个极简组合逻辑前端展示 Haskell 的类型建模、递归变换、错误处理、parser 组合、状态传递和代码生成。

展示时不要把重点放在资料来源上。真正应该讲清楚的是：为什么这些 Haskell 知识点能自然对应到 EDA 前端流程。

1. 技术流程

当前 demo 的流程如下。它把一个布尔表达式或多输出组合逻辑设计，转换成 AST，再进行仿真、化简、等价性检查和代码生成。

```
■■■■■ / ■■■■■  
-> Parser Combinators  
-> Expr AST  
-> eval / truth table  
-> simplify  
-> exhaustive equivalence check  
-> State-style netlist  
-> Verilog-style assign
```

步骤	Haskell 知识	为什么适合
解析文本	Parser Combinators	小 parser 可以组合成完整表达式 parser
表示电路	ADT	电路表达式天然树，适合用递归数据类型
仿真求值	递归 + Either	递归遍历 AST；未知信号通过类型显式报错
逻辑化简	模式匹配	按 AST 形状写布尔代数重写规则
等价性检查	List processing	枚举输入组合，比较化简前后输出
生成 netlist	State 思想	自动生成 n1、n2 等中间 wire 名称
输出代码	纯函数转换	把 AST 转成 Verilog-style 字符串

2. ADT：用类型表示电路结构

ADT 是整个 demo 的核心。我们不是用字符串保存电路，而是用 `Expr` 表示电路 AST。这样每一种门都是一种构造器，结构由类型保证。

```
data Expr
  = Lit Bool
  | Var String
  | Not Expr
  | And Expr Expr
  | Or Expr Expr
  | Xor Expr Expr
  | Nand Expr Expr
  | Nor Expr Expr
```

为什么能这样用：组合逻辑表达式天然就是递归树。变量和常量是叶子节点，NOT 是一元节点，AND/OR/XOR/NAND/NOR 是二元节点。

实际好处：求值、化简、统计门数、生成 netlist 和生成 Verilog-style 输出，都可以围绕同一个 AST 展开。

构造器	电路含义
Var String	输入信号或 wire
Not Expr	反相器
And / Or / Xor	常见组合逻辑门
Nand / Nor	常见功能完备门

3. 递归、模式匹配和 Either

递归和模式匹配负责处理 AST。`Either` 负责把失败写进类型。

```
eval :: Env -> Expr -> Either String Bool
eval env (Var x) =
  case Map.lookup x env of
    Just b  -> Right b
    Nothing -> Left ("unknown signal: " ++ x)
eval env (Not e)  = not <$> eval env e
eval env (And a b) = (&&) <$> eval env a <*> eval env b
eval env (Xor a b) = (/=) <$> eval env a <*> eval env b
```

为什么能这样用：每个电路节点的语义都可以按节点形状定义。`And a b` 的语义就是先求 `a` 和 `b`，再做 `&&`。如果某个子表达式失败，`Either` 会传播错误。

实际好处：仿真不会默默吞掉未知信号。比如缺少输入 `a` 时，程序会输出 `unknown signal: a`。

4. Parser Combinators

parser combinator 的思想是把小解析器组合成大解析器。这个项目支持 NOT、AND、OR、XOR、NAND、NOR、括号、变量和常量。

```
parseOr  = chainl1 parseXor orOp
parseXor = chainl1 parseAnd xorOp
parseAnd = chainl1 parseNot andOp
parseNot = NOT parser <|> atom parser
```

为什么能这样用：表达式语法本身有层级和优先级。把 OR、XOR、AND、NOT 分层解析，可以自然表达优先级。

实际好处：输入从手写 AST 升级成可读的 `.logic` 文件，例如 full adder：

```
sum = a XOR b XOR cin
carry = (a AND b) OR (cin AND (a XOR b))
```

5. State 思想和代码生成

生成 netlist 时需要自动生成中间 wire 名称, 例如 `n1`、`n2`。这就是状态问题。

```
newtype SimpleState s a =  
  SimpleState { runSimpleState :: s -> (a, s) }
```

```
fresh :: SimpleState NetState String  
fresh = SimpleState $ \(n, linesOut) ->  
  ("n" ++ show n, (n + 1, linesOut))
```

为什么能这样用: State 的本质是 `旧状态 -> 结果 + 新状态`。这里旧状态是当前编号和已经生成的 netlist 行。

实际输出例子:

```
n1 = XOR a b  
n2 = XOR n1 cin  
sum = BUF n2
```

同一个 AST 也可以转成 Verilog-style 输出:

```
assign sum = ((a ^ b) ^ cin);  
assign carry = ((a & b) | (cin & (a ^ b)));
```

6. 生产意义和边界

这个 demo 本身不是生产工具，但它对应生产级 EDA 前端里的真实思想：解析、AST/IR、语义分析、优化、等价性检查、netlist/code generation。

工业界现在按场景使用不同语言。Haskell 不是主流 RTL 语言，它的优势更接近生成器、转换器、检查器和参考模型。

场景	工业界常用语言
RTL 设计	Verilog / SystemVerilog / VHDL
验证 testbench	SystemVerilog + UVM, Python cocotb
系统级建模 / HLS	C / C++ / SystemC, MATLAB/Simulink
脚本自动化	Tcl、Python、Perl、Shell
EDA 工具内部	大量 C / C++，再配合脚本语言
硬件生成器 / 新型 HDL	Scala Chisel、SpinalHDL、Python Amaranth、Haskell Clash

所以我们不能说 Haskell 比 SystemVerilog 更适合手写生产 RTL。更准确的说法是：Haskell 适合把电路当成结构来生成、转换、检查。

问题	回答
生产中能直接用吗	不能。规模、语法、验证和工程可靠性都不够。
有没有实际意义	有。它是内部 DSL、RTL 生成器、验证辅助工具、golden model 的最小原型。
为什么用 Haskell	Haskell 的 ADT、模式匹配、纯函数和 Monad 很适合结构化变换。
舍弃了什么	完整 Verilog 解析、时序逻辑、位宽、复杂优化、GUI、工业级验证。
下一步	支持 Verilog 子集输入、标准 wire 声明、QuickCheck/SAT 验证。

推荐答法

我们没有声称做了生产级 EDA 工具。我们做的是 EDA 前端核心机制的最小可运行原型，用来展示 Haskell 知识点为什么适合这类结构化程序变换。